Step 4 — Forgetting, and a Privacy-Respecting Delete — Solution

> Back to the step · Hint 1 · Hint 2

Story: a system that remembers a user must be genuinely able to forget them too, completely, not just partially. `forget_user` deletes **every** stored item in that user's namespace — not just one known key — so a fresh conversation afterward truly knows nothing about them. **If not:** a "forget me" that clears only the name while quietly leaving preferences and other facts behind is a privacy promise you only half kept.

`src/memory/store.py` — add `forget_user`

Story: this lists every item actually sitting in the user's namespace and deletes each one individually. Step 3 may have written several different keys over time (`preferred_name` , a dietary preference, and more), so a delete that only clears one known key isn't a real forget at all. **If not:** stale facts would survive the "delete" and quietly resurface in the user's very next conversation.

```
# src/memory/store.py  (add this)
def forget_user(store, user_id):
    namespace = namespace_for(user_id)
    deleted = 0
    for item in store.search(namespace):
        store.delete(namespace, item.key)
        deleted = deleted + 1
    return deleted
```

`main.py` — the `forget_me` command

```
# in main.py's loop, before the normal invoke:
from src.memory.store import forget_user
# ...
        if text == "forget_me":
            count = forget_user(store, "sam")
            print(f"MemoryKeeper: forgotten — removed {count} saved facts.")
            continue
```

After `forget_me`, a brand-new thread's `recall_node` finds nothing, and the model has no idea who you are.

`tests/unit/test_store.py` — add the forget case

```
# add to tests/unit/test_store.py
from src.memory.store import forget_user


def test_forget_removes_all():
    store = build_store()
    save_fact(store, "sam", "preferred_name", "Name is Sam")
    save_fact(store, "sam", "seat", "Likes window seats")
    removed = forget_user(store, "sam")
    result = "OK"
    if removed != 2:
        result = "WRONG"
    if recall_facts(store, "sam") != []:
        result = "WRONG"
    print(f"case 4 (forget removes all)     -> {result}")


# and call it under __main__:
#     test_forget_removes_all()
```

Expected output (all four `test_store` cases):

```
case 1 (save then recall)      -> OK
case 2 (another user sees none) -> OK
case 3 (same key updates)       -> OK
case 4 (forget removes all)     -> OK
```

The integration test (the whole memory story, live)

```
# tests/integration/test_live_memory.py
# The real proof with the model. Run from the repo root:
#   python -m tests.integration.test_live_memory
from langchain_core.messages import HumanMessage
from langgraph.checkpoint.memory import MemorySaver

from src.config import load_config
from src.graph import build_graph
```

```
from src.memory.store import build_store, forget_user, recall_facts
from src.models.llm import get_model

USER = "sam"


def fresh_app():
    config = load_config()
    model = get_model(config)
    store = build_store()
    app = build_graph(model, store, MemorySaver())
    return app, store


def say(app, text, thread):
    out = app.invoke(
        {"messages": [HumanMessage(content=text)], "user_id": USER,
         "recalled": ""},
        {"configurable": {"thread_id": thread}},
    )
    return out["messages"][-1].content


def test_remembers_across_threads():
    app, store = fresh_app()
    say(app, "Hi, my name is Sam.", "t1")
    answer = say(app, "What's my name?", "brand-new-thread")
    result = "OK"
    if "sam" not in answer.lower():
        result = "WRONG"
    print(f"case 1 (recalls name in a new thread) -> {result}")


def test_ephemeral_not_saved():
    app, store = fresh_app()
    say(app, "I think I'll have pasta tonight.", "t1")
    result = "OK"
    if recall_facts(store, USER) != []:
        result = "WRONG"
    print(f"case 2 (ephemeral not remembered)     -> {result}")


def test_rename_updates():
    app, store = fresh_app()
    say(app, "My name is Sam.", "t1")
    say(app, "Actually, call me Sammy.", "t2")
    facts = recall_facts(store, USER)
```

```
    result = "OK"
    name_facts = []
    for fact in facts:
        if "name" in fact.lower():
            name_facts.append(fact)
    if len(name_facts) != 1 or "sammy" not in name_facts[0].lower():
        result = "WRONG"
    print(f"case 3 (rename updates, not dup)      -> {result}")


def test_forget_leaves_nothing():
    app, store = fresh_app()
    say(app, "My name is Sam.", "t1")
    forget_user(store, USER)
    answer = say(app, "What's my name?", "after-forget")
    result = "OK"
    if "sam" in answer.lower():
        result = "WRONG"
    if recall_facts(store, USER) != []:
        result = "WRONG"
    print(f"case 4 (forget -> knows nothing)      -> {result}")


if __name__ == "__main__":
    test_remembers_across_threads()
    test_ephemeral_not_saved()
    test_rename_updates()
    test_forget_leaves_nothing()
```

Expected output (real model):

```
case 1 (recalls name in a new thread) -> OK
case 2 (ephemeral not remembered)     -> OK
case 3 (rename updates, not dup)      -> OK
case 4 (forget -> knows nothing)      -> OK
```

> **A note on persistence:** the checkpointer is `SqliteSaver`, so a specific conversation (`thread_id`) survives a restart. The Store here is `InMemoryStore`, so user facts live as long as the assistant process does — long enough to prove the two-memory architecture and the cross-thread recall. For user memory that also survives a process restart, swap `InMemoryStore` for a file- or database-backed store with the same `put` / `get` / `search` / `delete` interface — every other line stays the same.

That's the whole project: a per-thread checkpointer and a per-user store, a brand-new conversation that still knows you, a system that decides for itself what's worth keeping, a rename that updates instead of duplicating, and a `forget_me` that truly forgets.